

✓ Practice Questions (30 total)

◆ 10 Lambda Expression Problems

1. Sort employees by name then age
Given a list of `Employee(name, age)`, use `List.sort()` with a lambda that compares first by name (ascending) and then by age (descending).

Solution

```
class Employee { String name; int age; }
List<Employee> list = ... ;
list.sort((e1, e2) -> {
    int nameComp = e1.name.compareTo(e2.name);
    if (nameComp != 0) return nameComp;
    return Integer.compare(e2.age, e1.age); // descending age
});
```

2. Remove all strings shorter than 4 characters
Use `List.removeIf()` with a lambda predicate.

Solution

```
List<String> words = new ArrayList<>(Arrays.asList("a", "code", "java", "is", "fun"));
words.removeIf(s -> s.length() < 4);
// [ "code", "java", "fun" ]
```

3. Transform a list of strings to a list of their lengths
Write a method that takes a `List<String>` and a `Function<String, Integer>` and returns a list of integers.
Call it with a lambda `s -> s.length()`.

Solution

```
public static List<Integer> map(List<String> list, Function<String, Integer> mapper) {
    List<Integer> result = new ArrayList<>();
    for (String s : list) result.add(mapper.apply(s));
    return result;
}
// usage: map(words, s -> s.length())
```

4. Sort strings by number of vowels
Use a lambda comparator that counts vowels in each string.

Solution

```
words.sort((a, b) -> countVowels(a) - countVowels(b));
// helper countVowels can be a method reference
```

5. Update all numbers in a list by replacing them with their absolute values
Use `List.replaceAll(UnaryOperator)` with a lambda.

Solution

```
List<Integer> nums = new ArrayList<>(Arrays.asList(-2, 5, -7, 0));
nums.replaceAll(x -> Math.abs(x));
```

6. Combine two maps by adding values for duplicate keys
Given two Map<String, Integer>, merge them using Map.merge() and a BiFunction lambda (v1, v2) -> v1 + v2.

Solution

```
Map<String, Integer> m1 = ..., m2 = ...;
m2.forEach((k, v) -> m1.merge(k, v, (oldVal, newVal) -> oldVal + newVal));
```

7. Generate a list of 10 random integers between 1 and 100
Use a Supplier<Integer> lambda passed to a method that fills a list.

Solution

```
public static List<Integer> generate(int n, Supplier<Integer> supplier) {
    List<Integer> list = new ArrayList<>();
    for (int i = 0; i < n; i++) list.add(supplier.get());
    return list;
}
// generate(10, () -> new Random().nextInt(100) + 1);
```

8. Find the maximum of two numbers using a BinaryOperator
Write a method int max(int a, int b, BinaryOperator<Integer> op) and pass a lambda (x, y) -> x > y ? x : y.

Solution

```
public static int max(int a, int b, BinaryOperator<Integer> op) { return op.apply(a, b); }
// max(3, 5, (x, y) -> x > y ? x : y)
```

9. Execute an action on each element of a list using a Consumer
Write a method void forEach(List<T> list, Consumer<T> consumer).
Use it to print each string in uppercase.

Solution

```
public static <T> void forEach(List<T> list, Consumer<T> consumer) {
    for (T item : list) consumer.accept(item);
}
// forEach(words, s -> System.out.println(s.toUpperCase()));
```

10. Filter a list using a Predicate
Write a generic filter method that takes a list and a predicate and returns a new list. Use it to keep only numbers > 10.

Solution

```
public static <T> List<T> filter(List<T> list, Predicate<T> predicate) {
    List<T> result = new ArrayList<>();
    for (T item : list) if (predicate.test(item)) result.add(item);
}
```

```

        return result;
    }
    // filter(nums, n -> n > 10);

```

◆ 10 Stream API (map / filter / reduce) Problems

1. Sum of squares of even numbers
List<Integer> nums → sum of squares of evens.

Solution

```

int sum = nums.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(n -> n * n)
    .sum();

```

2. Find the most frequent word
From List<String> words, return the word with highest frequency. Use streams + groupingBy.

Solution

```

String mostFreq = words.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey)
    .orElse(null);

```

3. Group strings by their length
Return Map<Integer, List<String>>.

Solution

```

Map<Integer, List<String>> grouped = words.stream()
    .collect(Collectors.groupingBy(String::length));

```

4. Partition numbers into even and odd
Use Collectors.partitioningBy.

Solution

```

Map<Boolean, List<Integer>> partitioned = nums.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));

```

5. Flatten a list of lists and sum all numbers
List<List<Integer>> → total sum using flatMapToInt.

Solution

```
int total = listOfLists.stream()
    .flatMapToInt(list -> list.stream().mapToInt(Integer::intValue))
    .sum();
```

6. Count distinct characters in a list of strings
Use flatMap to stream characters and distinct().

Solution

```
long distinctCount = words.stream()
    .flatMap(s -> s.chars().mapToObj(c -> (char) c))
    .distinct()
    .count();
```

7. Find the second highest number
Assume all numbers distinct.

Solution

```
int secondMax = nums.stream()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst()
    .orElseThrow(() -> new RuntimeException("No second element"));
```

8. Concatenate strings with comma (no leading/trailing comma)
Using Collectors.joining(", ").

Solution

```
String joined = words.stream().collect(Collectors.joining(", "));
```

9. Average of numbers greater than a threshold
double avg = nums.stream().filter(n -> n > threshold).mapToDouble(n -> n).average().orElse(0.0);

Solution

```
double avg = nums.stream()
    .filter(n -> n > 10)
    .mapToDouble(n -> n)
    .average()
    .orElse(0.0);
```

10. Check if any string is palindrome
Use anyMatch.

Solution

```
boolean hasPalindrome = words.stream()
    .anyMatch(s -> s.equals(new StringBuilder(s).reverse().toString()));
```

◆ 10 Remaining Topics (Optional, Method References, Functional Interfaces)

1. Avoid NPE when accessing nested object

Given Person with Address with City, return city name or "Unknown" safely using Optional.

Solution

```
public String getCityName(Person person) {  
    return Optional.ofNullable(person)  
        .map(Person::getAddress)  
        .map(Address::getCity)  
        .orElse("Unknown");  
}
```

2. Transform Optional value

Given Optional<Integer>, return the square if present, else Optional.empty().

Solution

```
Optional<Integer> squared = opt.map(x -> x * x);
```

3. Provide lazy default value

Use orElseGet with a supplier to compute default only when needed.

Solution

```
String value = optional.orElseGet(() -> expensiveDefault());
```

4. Chain Optionals with flatMap

If you have an Optional<User> and want to get the user's email if it's present and nonEmpty, use flatMap.

Solution

```
Optional<String> email = optionalUser  
    .flatMap(User::getEmail)           // returns Optional<String>  
    .filter(e -> !e.isEmpty());
```

5. Sort employees by age using method reference

Comparator.comparingInt(Employee::getAge).

Solution

```
employees.sort(Comparator.comparingInt(Employee::getAge));
```

6. Convert strings to integers using method reference

Use Integer::parseInt with map.

Solution

```
List<Integer> ints = strings.stream()  
    .map(Integer::parseInt)  
    .collect(Collectors.toList());
```

7. Create objects with constructor reference
Given `List<String> names`, create `List<Student>` using `Student::new` if constructor `Student(String name)` exists.

Solution

```
List<Student> students = names.stream()
    .map(Student::new)
    .collect(Collectors.toList());
```

8. Custom functional interface: `StringFormatter`
Define `@FunctionalInterface` interface `StringFormatter { String format(String original); }` and implement with lambda to wrap in [].

Solution

```
@FunctionalInterface
interface StringFormatter { String format(String s); }
StringFormatter bracket = s -> "[" + s + "]";
System.out.println(bracket.format("hello")); // [hello]
```

9. Predicate method reference for non-empty strings
Use `String::isEmpty` negated with `Predicate.not()`.

Solution

```
Predicate<String> nonEmpty = Predicate.not(String::isEmpty);
```

10. Function method reference to get length
Use `String::length` as a function.

Solution

```
Function<String, Integer> lengthFn = String::length;
List<Integer> lengths = words.stream().map(String::length).collect(Collectors.toList());
```

Scenario-Based Exercises with Driver Code

Each exercise includes: - A problem statement - An empty method you must implement - A `main()` with embedded test cases - Automatic verification: "All test cases passed" or "X test cases failed"

Exercise 1: Stream – Process Orders and Compute Total Revenue

Problem:

You have a list of `Order` objects. Each order has: - `String customer` - `double amount` - `boolean isShipped`

Use streams to compute the total revenue from shipped orders where the amount is at least 50.0.

Implement `totalShippedRevenue(List<Order> orders)`.

Driver Code:

```
import java.util.*;

class Order {
    String customer;
    double amount;
    boolean isShipped;

    Order(String c, double a, boolean s) {
        customer = c; amount = a; isShipped = s;
    }
}

public class OrderProcessor {
    public static double totalShippedRevenue(List<Order> orders) {
        // TODO: use stream filter, mapToDouble, sum
        return 0.0;
    }

    public static void main(String[] args) {
        List<Order> test1 = Arrays.asList(
            new Order("Alice", 100.0, true),
            new Order("Bob", 49.99, true),
            new Order("Carol", 150.0, false),
            new Order("Dave", 200.0, true)
        );
        double expected1 = 100.0 + 200.0; // 300.0
        List<Order> test2 = Collections.emptyList();
        double expected2 = 0.0;
        List<Order> test3 = Arrays.asList(
            new Order("Eve", 30.0, true),
            new Order("Frank", 30.0, false)
        );
        double expected3 = 0.0;

        Object[][] tests = {
            {test1, expected1},
            {test2, expected2},
            {test3, expected3}
        };
    }
}
```

```

    int passed = 0;
    for (int i = 0; i < tests.length; i++) {
        List<Order> orders = (List<Order>) tests[i][0];
        double expected = (double) tests[i][1];
        double result = totalShippedRevenue(orders);
        if (Math.abs(expected - result) < 0.0001) {
            passed++;
            System.out.println("Test " + (i+1) + " passed");
        } else {
            System.out.println("Test " + (i+1) + " FAILED");
            System.out.println("Expected: " + expected + " but got: " + result);
        }
    }
    System.out.println(passed + "/" + tests.length + " tests passed");
}
}

```

Exercise 2: Optional – Safe Retrieval from a Nested Structure

Problem:

You have a `Company` that has an `Address` which has a `String` `city`.
 Implement `getCityName(Company company)` that returns the city name if present, otherwise "No City".
 Use `Optional` to chain the calls safely.

Driver Code:

```

class Address {
    String city;
    Address(String c) { city = c; }
    String getCity() { return city; }
}

class Company {
    String name;
    Address address; // may be null
    Company(String n, Address a) { name = n; address = a; }
    Address getAddress() { return address; }
}

public class CityFinder {
    public static String getCityName(Company company) {
        // TODO: implement using Optional
        return "";
    }
}

```



```

public static void main(String[] args) {
    Address addr1 = new Address("Pune");
    Company comp1 = new Company("Tech", addr1);
    Company comp2 = new Company("Biz", null);
    Company comp3 = null;

    Object[] [] tests = {
        {comp1, "Pune"},
        {comp2, "No City"},
        {comp3, "No City"}
    };

    int passed = 0;
    for (int i = 0; i < tests.length; i++) {
        Company input = (Company) tests[i][0];
        String expected = (String) tests[i][1];
        String result = getCityName(input);
        if (expected.equals(result)) {
            passed++;
            System.out.println("Test " + (i+1) + " passed");
        } else {
            System.out.println("Test " + (i+1) + " FAILED");
            System.out.println("Expected: " + expected + " but got: " + result);
        }
    }
    System.out.println(passed + "/" + tests.length + " tests passed");
}

```

Exercise 3: Lambda + Custom Functional Interface – String Transformation Pipeline

Problem:

Define a functional interface `StringProcessor` with method `String process(String input)`.

Implement a method `applyTransformations` that takes a `List<String>` and a `StringProcessor` and returns a new list with the transformation applied to each element.

The driver will test with:

- converting to uppercase
- reversing each string
- wrapping in parentheses

Driver Code:

```

@FunctionalInterface
interface StringProcessor {
    String process(String input);
}

public class StringTransformer {
    public static List<String> applyTransformations(List<String> list, StringProcessor proc) {
        // TODO: implement using the functional interface
        return new ArrayList<>();
    }

    public static void main(String[] args) {
        List<String> words = Arrays.asList("java", "lambda", "stream");

        // Test 1: uppercase
        StringProcessor toUpper = s -> s.toUpperCase();
        List<String> expectedUpper = Arrays.asList("JAVA", "LAMBDA", "STREAM");

        // Test 2: reverse
        StringProcessor reverse = s -> new StringBuilder(s).reverse().toString();
        List<String> expectedReverse = Arrays.asList("avaj", "adbmal", "maerts");

        // Test 3: brackets
        StringProcessor bracket = s -> "(" + s + ")";
        List<String> expectedBracket = Arrays.asList("(java)", "(lambda)", "(stream)");

        // Test 4: empty list
        List<String> empty = Collections.emptyList();

        Object[][] tests = {
            {words, toUpper, expectedUpper},
            {words, reverse, expectedReverse},
            {words, bracket, expectedBracket},
            {empty, toUpper, empty}
        };

        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            List<String> input = (List<String>) tests[i][0];
            StringProcessor proc = (StringProcessor) tests[i][1];
            List<String> expected = (List<String>) tests[i][2];
            List<String> result = applyTransformations(input, proc);
            if (expected.equals(result)) {
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {

```

```

        System.out.println("Test " + (i+1) + " FAILED");
        System.out.println("Expected: " + expected + " but got: " + result);
    }
}
System.out.println(passed + "/" + tests.length + " tests passed");
}
}

```

Exercise 4: Stream + Method References – Student Grade Analyser

Problem:

A Student has `String` `name` and `int` `score`.

Implement `getTopStudents(List<Student> students, int threshold)` that returns a list of names of students whose score is greater than the threshold, sorted by score descending, and in case of tie, by name alphabetically. Use method references where possible (e.g., `Student::getScore`).

Driver Code:

```

import java.util.*;
import java.util.stream.Collectors;

class Student {
    String name;
    int score;
    Student(String n, int s) { name = n; score = s; }
    String getName() { return name; }
    int getScore() { return score; }
}

public class GradeAnalyser {
    public static List<String> getTopStudents(List<Student> students, int threshold) {
        // TODO: stream, filter, sort (using Comparator.comparing), map, collect
        return new ArrayList<>();
    }

    public static void main(String[] args) {
        List<Student> all = Arrays.asList(
            new Student("Alice", 85),
            new Student("Bob", 92),
            new Student("Charlie", 85),
            new Student("David", 78),
            new Student("Eve", 92)
        );
    }
}

```

```

// threshold 80
List<String> expected1 = Arrays.asList("Bob", "Eve", "Alice", "Charlie");
// Bob (92), Eve (92), Alice (85), Charlie (85) - Eve before Alice? tie by score 92
// Actually expected order: score desc, name asc -> Bob(92), Eve(92) (Bob before Eve)
// Let's fix expected: Bob, Eve, Alice, Charlie (correct)
// threshold 90
List<String> expected2 = Arrays.asList("Bob", "Eve");
// threshold 95
List<String> expected3 = Collections.emptyList();

Object[] [] tests = {
    {all, 80, expected1},
    {all, 90, expected2},
    {all, 95, expected3}
};

int passed = 0;
for (int i = 0; i < tests.length; i++) {
    List<Student> students = (List<Student>) tests[i][0];
    int thresh = (int) tests[i][1];
    List<String> expected = (List<String>) tests[i][2];
    List<String> result = getTopStudents(students, thresh);
    if (expected.equals(result)) {
        passed++;
        System.out.println("Test " + (i+1) + " passed");
    } else {
        System.out.println("Test " + (i+1) + " FAILED");
        System.out.println("Expected: " + expected + " but got: " + result);
    }
}
System.out.println(passed + "/" + tests.length + " tests passed");
}
}

```

All driver code is self-contained and ready to run. Just fill in the empty methods and verify your implementation automatically.